

TD/TP 4 — Copie et déplacement de IntArray

Faire se comporter une classe comme un type natif

MAM4 — Programmation C++ Durée : 2 heures

Objectif. Compléter la classe RAIH de la séance 3. Une copie devra créer un tableau indépendant ; un déplacement devra transférer le tableau sans recopier ses éléments. La dernière partie montrera comment retrouver une interface proche de celle de `std::vector`.

Le point de départ est important : à la fin de la séance 3, la copie a été interdite avec `= delete`. Le premier exercice la réactive brièvement pour observer le problème, puis les exercices suivants la définissent correctement.

Les sources sont fournies dans `code-td-seance-04.zip`. Après extraction, compiler et exécuter avec :

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    int_array.cpp main.cpp -o int_array
./int_array
```

Conserver les sanitizers pendant toute la séance. Les exercices 1 à 6 forment le parcours principal ; la dernière section est facultative.

Exercice 1 — Pourquoi la copie automatique est-elle incorrecte ? (15 min)

Objectif concret. Observer ce que le compilateur copie lorsqu'une classe contient un pointeur propriétaire.

Dans `int_array.hpp`, commenter provisoirement :

```
IntArray(const IntArray&) = delete;
IntArray& operator=(const IntArray&) = delete;
```

Ajouter ce test dans une portée séparée :

```
{
    IntArray a{4};
    a.push_back(1);

    IntArray b{a};
    b.fill(9);

    assert(a.at(0) == 9);
}
```

Avant d'exécuter, prédire :

1. Le compilateur crée-t-il un second tableau dynamique ?
2. Pourquoi la modification effectuée par `b.fill(9)` est-elle également visible depuis `a` ?
3. Que feront les deux destructeurs à la sortie du bloc ?

Exécuter avec AddressSanitizer et relever le diagnostic essentiel. Retirer ce bloc de diagnostic, puis restaurer les deux lignes `= delete`.

À retenir. La copie automatique recopie la valeur de `data_`, donc une adresse. Copier l'adresse ne crée pas un second tableau.

Exercice 2 — Construire une copie indépendante (20 min)

Objectif concret. Obtenir deux objets de même contenu qui peuvent ensuite évoluer indépendamment.

Remplacer la première interdiction de copie par :

```
IntArray(const IntArray& other);
```

Définir le constructeur en suivant cet ordre :

1. reprendre la capacité et la taille de `other` ;
2. créer une nouvelle allocation ;
3. recopier les `size_` éléments utiles.

La classe admet aussi l'état interne vide `0 / 0 / nullptr`. Si `other.capacity_ == 0`, conserver un pointeur nul. On pourra utiliser `std::copy_n` après avoir inclus `<algorithm>`.

Tester immédiatement :

```
IntArray a{4};
a.push_back(1);

IntArray b{a};
b.fill(9);

assert(a.at(0) == 1);
assert(b.at(0) == 9);
assert(a.size() == b.size());
assert(a.capacity() == b.capacity());
```

Question. Quelle instruction du constructeur garantit que **a** et **b** ne possèdent pas la même allocation ? Quel est le coût de cette copie pour un tableau contenant n éléments ?

À retenir. Une copie correcte possède le même contenu observable, mais une ressource indépendante.

Exercice 3 — Affecter un objet déjà construit (25 min)

Objectif concret. Remplacer le contenu d'un objet sans perdre trop tôt sa ressource actuelle.

Le constructeur de copie crée un nouvel objet. Dans **a = b**, au contraire, **a** existe déjà et possède peut-être un tableau. Avant d'écrire le code, ordonner les opérations suivantes :

allouer le nouveau tableau copier les éléments libérer l'ancien tableau installer le nouveau pointeur

Remplacer la seconde ligne **= delete** par :

```
IntArray& operator=(const IntArray& other);
```

L'opérateur doit :

- laisser **a = a** sans effet ;
- créer et remplir le remplacement avant de libérer l'ancien tableau ;
- accepter des capacités différentes ;
- renvoyer ***this**.

Tester :

```
IntArray a{2};
a.push_back(1);

IntArray b{8};
b.push_back(7);

a = b;
assert(a.size() == b.size());
assert(a.at(0) == 7);

a.fill(0);
assert(b.at(0) == 7);

a = a;
assert(a.at(0) == 0);
```

Question. Pourquoi ne faut-il pas commencer par **delete[] data_** ?

À retenir. Lors d'une affectation, préparer le remplacement avant de détruire l'ancienne ressource.

Exercice 4 — Transférer la ressource (30 min)

Objectif concret. Déplacer un `IntArray` en transférant seulement ses trois champs.

a) Constructeur de déplacement. Ajouter :

```
IntArray(IntArray&& other) noexcept;
```

Utiliser `std::exchange` pour prendre les valeurs de `other` et les remplacer par l'état vide canonique :

```
capacity_ = 0    size_ = 0    data_ = nullptr
```

Inclure `<utility>` dans les fichiers qui utilisent `std::exchange` ou `std::move`, puis tester :

```
IntArray a{4};
a.push_back(7);

IntArray b{std::move(a)};

assert(a.size() == 0);
assert(a.capacity() == 0);
assert(b.at(0) == 7);
```

Question. Le constructeur a-t-il recopié les éléments du tableau ? Qui possède maintenant l'allocation ? Pourquoi cette opération peut-elle être en $O(1)$?

b) Affectation par déplacement. Ajouter :

```
IntArray& operator=(IntArray&& other) noexcept;
```

L'objet de gauche existe déjà. Il doit libérer sa ressource, prendre les trois champs de `other`, vider `other`, puis renvoyer `*this`. Traiter aussi `a = std::move(a)`.

```
IntArray c{2};
c.push_back(99);

c = std::move(b);

assert(b.size() == 0);
assert(b.capacity() == 0);
assert(c.at(0) == 7);
```

c) Réutiliser la source. Notre contrat autorise la réutilisation d'un objet déplacé. Dans `push_back`, adapter le calcul de la nouvelle capacité :

```
const std::size_t new_capacity{
    capacity_ == 0 ? 1 : 2 * capacity_
};
```

Vérifier ensuite :

```
a.push_back(3);
assert(a.size() == 1);
assert(a.at(0) == 3);

IntArray empty_copy{b};
assert(empty_copy.size() == 0);
assert(empty_copy.capacity() == 0);
```

Question. Que fait réellement `std::move(a)` ? Quelle fonction réalise le transfert de la ressource ?

À retenir. Déplacer ne recopie pas le tableau : le constructeur ou l'opérateur de déplacement transfère sa propriété. `std::move` permet de sélectionner cette opération.

Exercice 5 — Retrouver l'accès naturel avec `operator[]` (15 min)

Objectif concret. Distinguer l'accès à un objet modifiable de l'accès à un objet constant.

Ajouter les deux surcharges :

```
int& operator[](std::size_t index);  
const int& operator[](std::size_t index) const;
```

Elles renvoient directement `data_[index]`. Contrairement à `at()`, `operator[]` ne vérifie pas les bornes.

```
IntArray values{4};  
values.push_back(3);  
  
values[0] = 42;  
assert(values[0] == 42);  
  
const IntArray& frozen{values};  
assert(frozen[0] == 42);  
// frozen[0] = 5; // erreur de compilation attendue
```

Activer séparément la dernière ligne, lire le diagnostic, puis la commenter de nouveau.

Question. Pourquoi la première surcharge renvoie-t-elle `int&` plutôt que `int` ? Pourquoi la seconde renvoie-t-elle `const int&` ?

Exercice 6 — Revenir à la règle de zéro (10 min)

Objectif concret. Identifier la solution que l'on préférerait dans du code applicatif.

Remplacer conceptuellement les trois membres :

```
std::size_t capacity_;  
std::size_t size_;  
int* data_;
```

par :

```
std::vector<int> data_;
```

Faire correspondre `size_`, `capacity_`, le stockage et `push_back` avec les opérations de `std::vector`. Pour réserver sans créer d'éléments, utiliser `data_.reserve(capacity_)`.

1. Quelles opérations spéciales écrites pendant le TD deviennent inutiles ?
2. Pourquoi les copies d'une classe contenant un `vector` sont-elles naturellement indépendantes ?
3. Résumer en deux phrases la règle des cinq et la règle de zéro.

À retenir. La règle des cinq permet de comprendre la gestion directe d'une ressource. Dans le code applicatif, on préfère généralement composer avec des types comme `std::vector` et bénéficier de la règle de zéro.

Pour aller plus loin — opérateurs `+=` et `+`

Cette partie reprend l'approfondissement de la fin du cours. Écrire d'abord l'opération sur place :

```
IntArray& operator+=(const IntArray& rhs);
```

Les deux tableaux doivent avoir la même taille; sinon lancer `std::invalid_argument`. Déclarer ensuite, après la classe :

```
IntArray operator+(IntArray lhs, const IntArray& rhs);
```

et définir cette fonction dans `int_array.cpp` en réutilisant `lhs += rhs`. Vérifier que `a += b` modifie `a`, tandis que `c = a + b` ne modifie ni `a` ni `b`.

Question. Pourquoi `lhs` est-il reçu par valeur ? Quel mécanisme écrit à l'exercice 2 rend cette solution correcte ?